Project Rebuild – Deployment Guide

Comprehensive, reproducible instructions for setting up, deploying, and testing the entire Project Rebuild smart-contract stack on Base Sepolia (testnet).

---

1. Prerequisites

1.1 Accounts & Keys

- MetaMask (or any EVM wallet) connected to Base Sepolia.
- Test wallet **private key** (never share your mainnet key).
- Coinbase/Alchemy/Infura account if you prefer their RPC endpoints.

1.2 Software

Tool	Version	Notes
Node.js	20.x	Required by Docker image
Docker Desktop	≥ 24.x	Needed for reproducible environment
Git	≥ 2.30	For dependency management

---

2. Environment Setup

2.1 Clone & Install

```
git clone <repo-url> project-rebuild
cd project-rebuild
npm install            # optional, handled in Docker but handy locally
```

2.2 Configure `.env`

```
cp .env.example .env
nano .env  # use any editor you prefer
```

Fill in:

```
BASE_SEPOLIA_RPC_URL=https://sepolia.base.org
PRIVATE_KEY=your_private_key_without_0x
FOUNDER_WALLET=0xYourWallet
VRF_SUBSCRIPTION_ID=<from Chainlink portal>
VRF_KEY_HASH=<from Chainlink portal>
AAVE_ATOKEN=0x0000000000000000000000000000000000000000    # optional
BASE_URI=https://api.projectrebuild.com/metadata/
BASE_IMAGE_URI=https://api.projectrebuild.com/images/
BASESCAN_API_KEY=<optional, for verification>
```

2.3 Fund Wallet & VRF

1. **Get Base Sepolia ETH:**
   https://www.coinbase.com/faucets/base-ethereum-goerli-faucet
   Need ≈0.1 ETH to deploy + interact.
2. **Create Chainlink VRF Subscription:**
   https://vrf.chain.link/base-sepolia
   - Create subscription
   - Note subscription ID + Key Hash
   - Add consumer later (after deployment)
   - Fund with ≥2 LINK.

---

3. Dockerized Tooling (Recommended)

3.1 Build Images

```
npm run docker:build       # full image
npm run docker:build:dev   # lighter dev image
```

3.2 Start Dev Node (optional)

```
npm run docker:node        # spins up Anvil-compatible node
```

3.3 Open Dev Shell

```
npm run docker:shell       # drops into /app with foundry installed
```

---

4. Pre-Deployment Validation

4.1 Install Foundry Dependencies (inside dev shell)

```
bash scripts/setup-dependencies.sh
```

4.2 Compile & Test

```
forge build
forge test --gas-report
```

4.3 Lint & Format (optional but recommended)

```
npm run lint
forge fmt
```

---

5. Deployment (Base Sepolia)

5.1 Interactive Script (preferred)

```
bash scripts/interactive-deploy.sh
```

The script:

1. Verifies `.env` values.
2. Checks wallet balance.
3. Deploys:
    - `ProjectRebuildNFT`
    - `TicketManager`

    - PrizePool
    - VRFConsumer
    - MetadataRenderer
    - AaveStaking
4. Shows all contract addresses.

5.2 Manual Command (if you prefer direct control)

```
forge script
contracts/deployments/DeployProjectRebuild.s.sol:DeployProjectRebuild \
  --rpc-url $BASE_SEPOLIA_RPC_URL \
  --broadcast \
  --verify \
  --etherscan-api-key $BASESCAN_API_KEY \
  -vvvv
```

5.3 Record Addresses

Update `.env` with the deployed addresses for easy reuse:

```
NFT_CONTRACT=0x...
TICKET_MANAGER=0x...
PRIZE_POOL=0x...
VRF_CONSUMER=0x...
METADATA_RENDERER=0x...
AAVE_STAKING=0x...
```

---

6. Post-Deployment Steps

6.1 Link VRF Consumer

1. Go to https://vrf.chain.link/base-sepolia.
2. Open your subscription → “Add consumer”.
3. Enter `VRF_CONSUMER` address and confirm.

6.2 Verify Contracts (optional but client-facing)

```
forge verify-contract \
```

```
  --chain-id 84532 \
  --num-of-optimizations 200 \
  --watch \
  --constructor-args $(cast abi-encode
"constructor(address,address,uint256)" \
      $USDC_BASE_SEPOLIA $FOUNDER_WALLET 667) \
  --etherscan-api-key $BASESCAN_API_KEY \
  $NFT_CONTRACT \
  contracts/core/ProjectRebuildNFT.sol:ProjectRebuildNFT
```

Repeat for other contracts if desired.

---

7. Contract Sanity Checks

```
bash scripts/test-deployed.sh
```

This script (uses `cast`) confirms:

- Owner address
- Total supply
- Mint price
- Prize pool balances

---

8. Minting Scenarios

8.1 Using Script

```
bash scripts/test-mint.sh
```

Prompts guide you through:

- Approving USDC
- Minting with/without referral
- Verifying referral earnings & tickets

8.2 Manual Commands

```
# Approve USDC (Base Sepolia USDC =
0x036CbD53842c5426634e7929541eC2318f3dCF7e)
```

```
cast send $USDC_BASE_SEPOLIA \
  "approve(address,uint256)" \
  $NFT_CONTRACT \
  15000000 \
  --rpc-url $BASE_SEPOLIA_RPC_URL \
  --private-key $PRIVATE_KEY

# Mint without referral
cast send $NFT_CONTRACT \
  "mint(address,uint256)" \
  $FOUNDER_WALLET \
  0 \
  --rpc-url $BASE_SEPOLIA_RPC_URL \
  --private-key $PRIVATE_KEY
```

8.3 Verification (Read Calls)

```
cast call $NFT_CONTRACT "totalSupply()"
cast call $NFT_CONTRACT "tokenURI(uint256)" 1
cast call $TICKET_MANAGER "getSmallBlockTickets(uint256)" 1
cast call $PRIZE_POOL "getSmallPoolBalance()"
```

---

9. Prize Pool Simulation

9.1 Batch Mint Script

```
./scripts/mint-batch.sh $NFT_CONTRACT $FOUNDER_WALLET $PRIVATE_KEY 500
$BASE_SEPOLIA_RPC_URL
```

500 mints → triggers $2,500 small pool target.

9.2 Monitor Events

```
cast logs --from-block latest --follow \
  "SmallBlockTriggered(uint256,uint256)" \
  --address $PRIZE_POOL \
  --rpc-url $BASE_SEPOLIA_RPC_URL
```

9.3 Process VRF Results

```
cast call $VRF_CONSUMER "getVRFRequest(uint256)" REQUEST_ID
cast send $NFT_CONTRACT "processBlockDraw(uint256)" REQUEST_ID \
  --rpc-url $BASE_SEPOLIA_RPC_URL \
  --private-key $PRIVATE_KEY
```

---

10. Metadata & Royalty Verification

10.1 Metadata

```
cast call $NFT_CONTRACT "tokenURI(uint256)" 1
# Decode base64 to verify referral stats, ticket counts, etc.
```

10.2 Refresh on Marketplaces

- On OpenSea Testnet → “Refresh metadata”.
- Confirm attributes update (referrals, tickets, badges).

10.3 Royalty Enforcement (ERC-721C)

1. Import contract on https://testnets.opensea.io.
2. Attempt 0% royalty listing → should fail/revert.
3. List with ≥6.67% royalty (≈$10 on $150 sale) → should succeed.

Verify `royaltyInfo`:

```
cast call $NFT_CONTRACT "royaltyInfo(uint256,uint256)" 1 15000000
```

4.

---

11. Full Test Matrix (Client Checklist)

Test	Goal	Command/Action	Expected Result
Compile	Ensure clean build	`forge build`	Success

Unit Tests	Cover core logic	`forge test -vvv`	All pass
Deployment	Base Sepolia	`bash scripts/interactive-deploy.sh`	Addresses logged
VRF Link	Randomness ready	Add consumer in Chainlink portal	Consumer added
Mint w/o Referral	Baseline mint	`test-mint.sh`	NFT minted, no referral payout
Mint w/ Referral	Ticket + payout	`test-mint.sh` (second half)	Tickets + $5 payout
Pool Increment	Prize accounting	`cast call $PRIZE_POOL "getSmallPoolBalance()"`	+$5 per mint
Prize Trigger	Random draw	500 mints or manual funding	`SmallBlockTriggered` event
Metadata	Dynamic attributes	`tokenURI` + OpenSea refresh	Referral stats update
Royalty Enforcement	ERC-721C compliance	OpenSea listing attempt	0% fails, ≥6.67% passes

13. Troubleshooting

Issue	Likely Cause	Fix
`forge` command missing	Dependencies not installed	Run `bash scripts/setup-dependencies.sh` in dev shell
Deployment revert	Insufficient ETH or wrong RPC	Check faucet, confirm `BASE_SEPOLIA_RPC_URL`

VRF not fulfilling	Consumer not added / no LINK	Add consumer in portal, fund subscription
USDC transfer fails	No allowance	Re-run `approve` before mint
Metadata stale	Cache on OpenSea	Use “Refresh metadata” button

---

14. Command Quick Reference

```
# Docker helpers
npm run docker:build
npm run docker:node
npm run docker:test
npm run docker:compile
npm run docker:shell

# Foundry basics
forge build
forge test -vvv
forge script <script> --rpc-url ... --broadcast -vvvv
forge verify-contract ...

# Cast essentials
cast call <contract> "<signature>" args
cast send <contract> "<signature>" args --private-key $PRIVATE_KEY
cast logs --address <contract> --from-block N --to-block latest
```

---

15. Support

If any step fails, capture:

1. Command executed
2. Full console output
3. Relevant transaction hash (if applicable)

Share that information for the fastest assistance.

You are now ready to deploy, validate, and demo Project Rebuild end-to-end on Base Sepolia.
Happy testing!